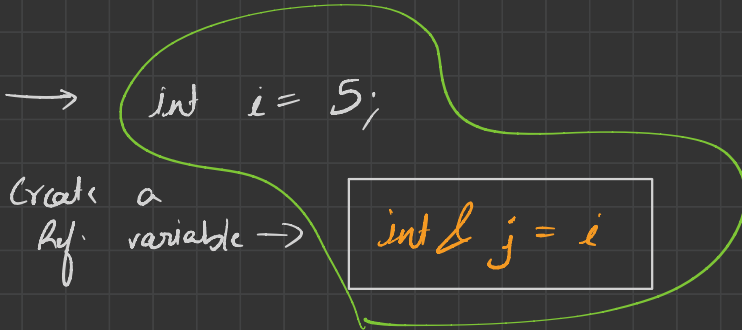
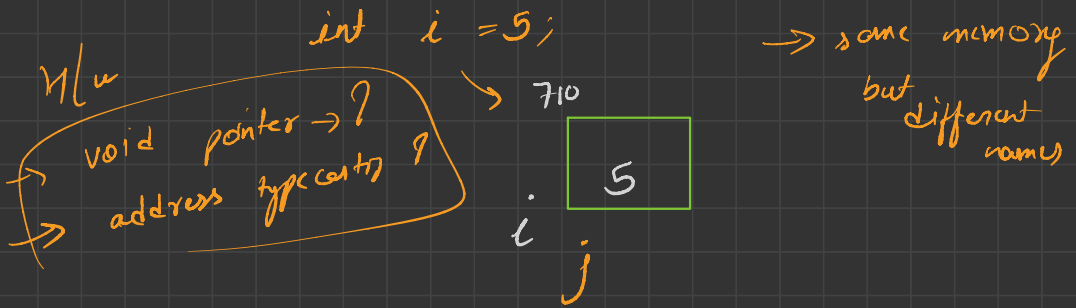
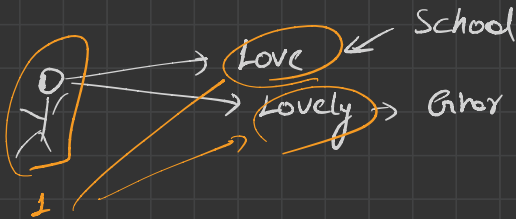
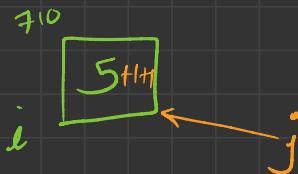



Lecture - 28

① Reference Variable

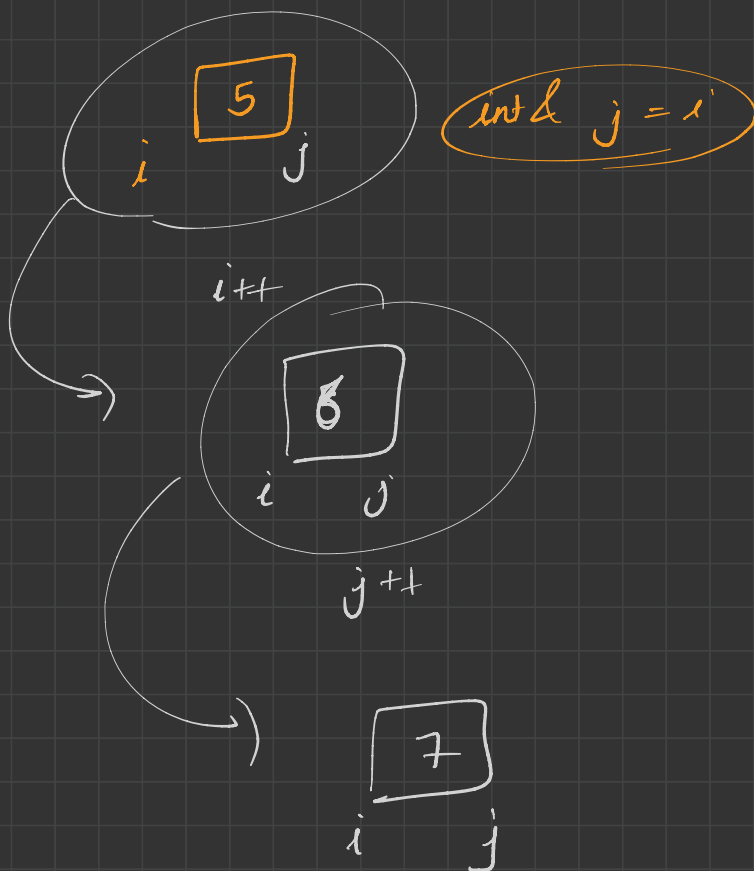


①



②

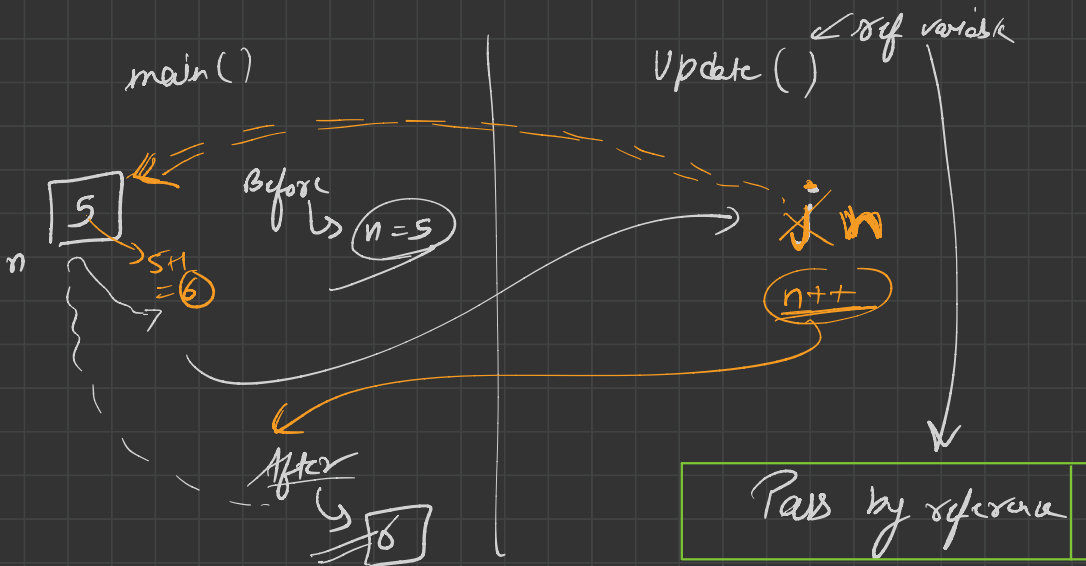
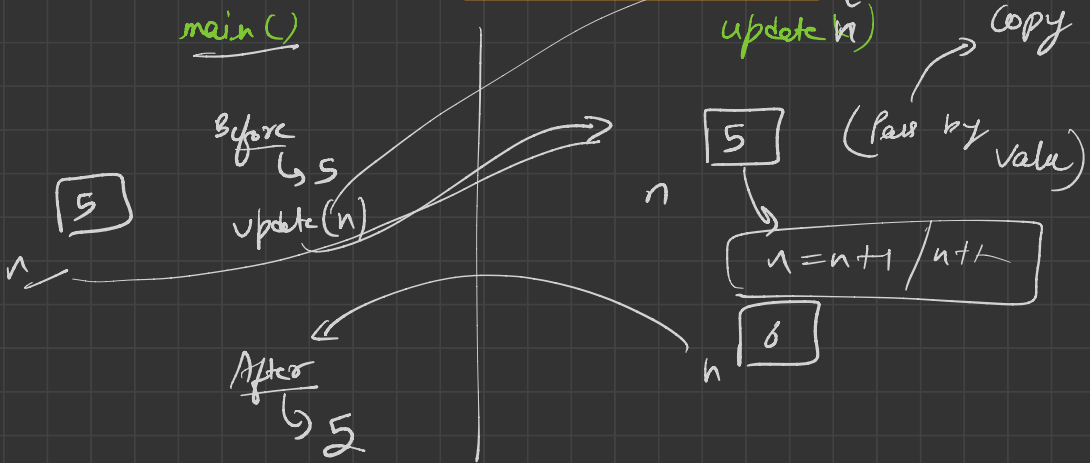
$i++$
 $j++$



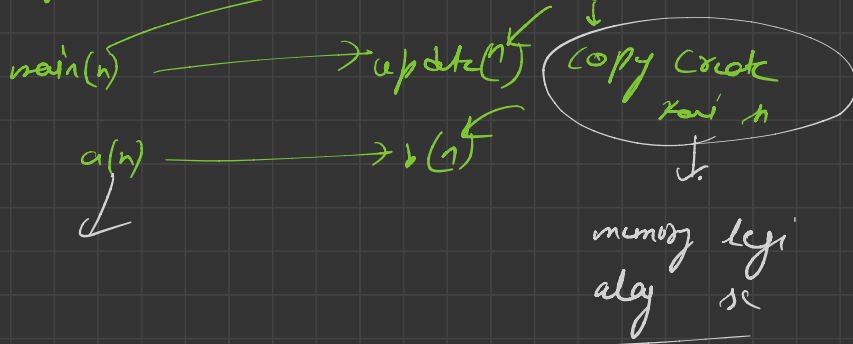
why - ?

① function

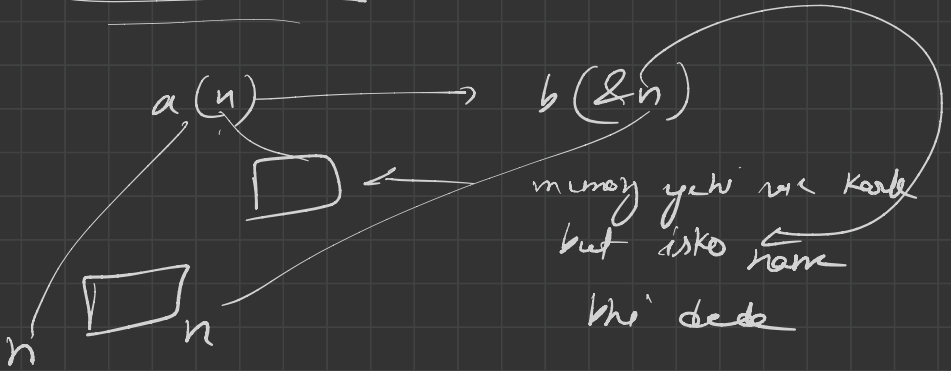
update (int n) ?
 $n++$;
3



Pass by value



Pass by reference



→ same memory
main different hai

void update (int n)

{

}

memory
access kr pao

→ pehle kisi or me
change kr dia

update (int n)

→ BAD

practice

int a = 10;

int ans = a;

return ans;

}

Ref var - ?

↳ Pass by value } → Diff
↳ Ref

↳ fun → Pass by ref

→ return by ref

Drawback

→ Array

int n;
cin >> n;

int arr[n];

runtime → Yes / No

→ BAD practice

compile time -

→ why?

int arr [10000];
→ 10000 → size

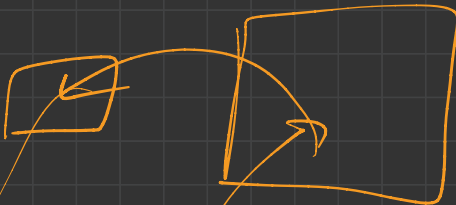
Program start

memory

stack

compile time

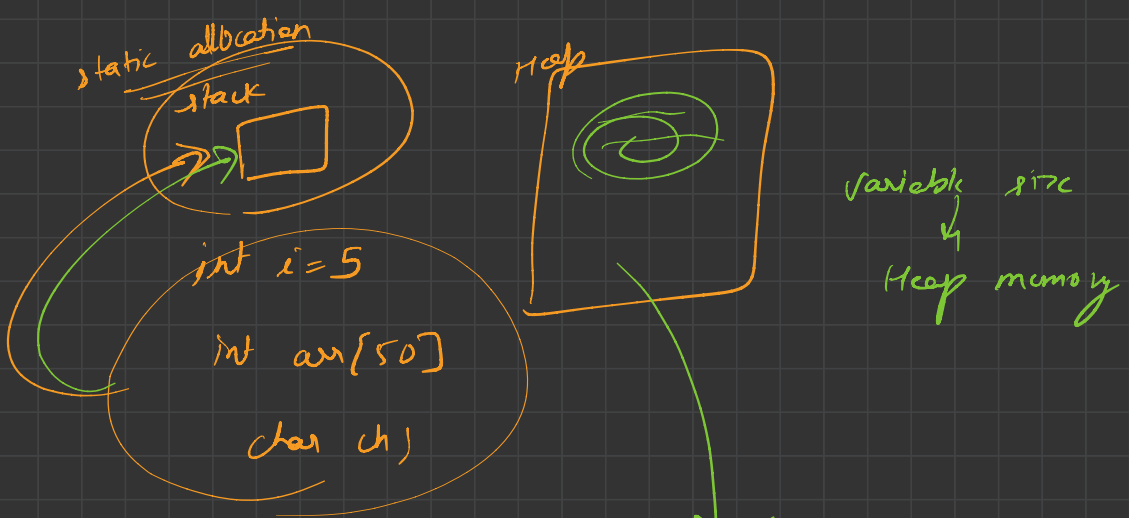
Heap



program crash

int n =
arr[n]

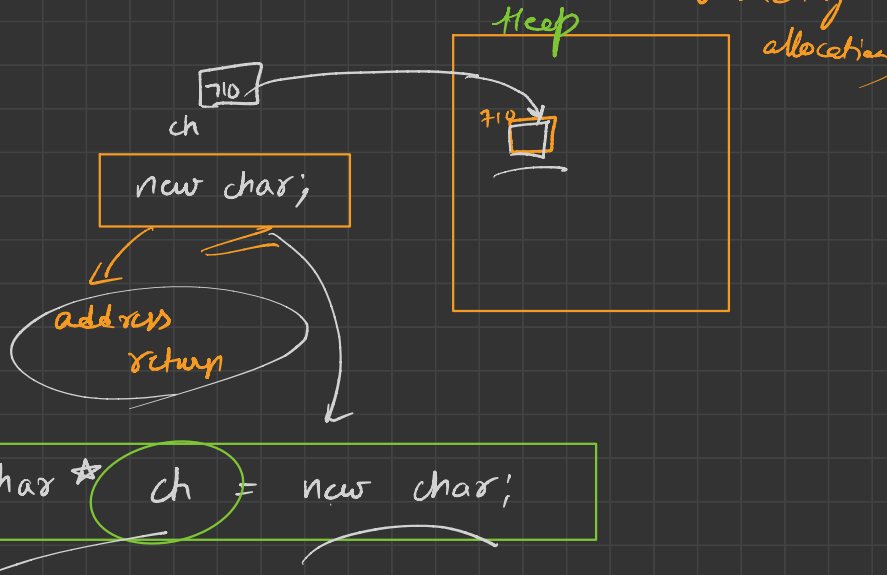
BAD practice

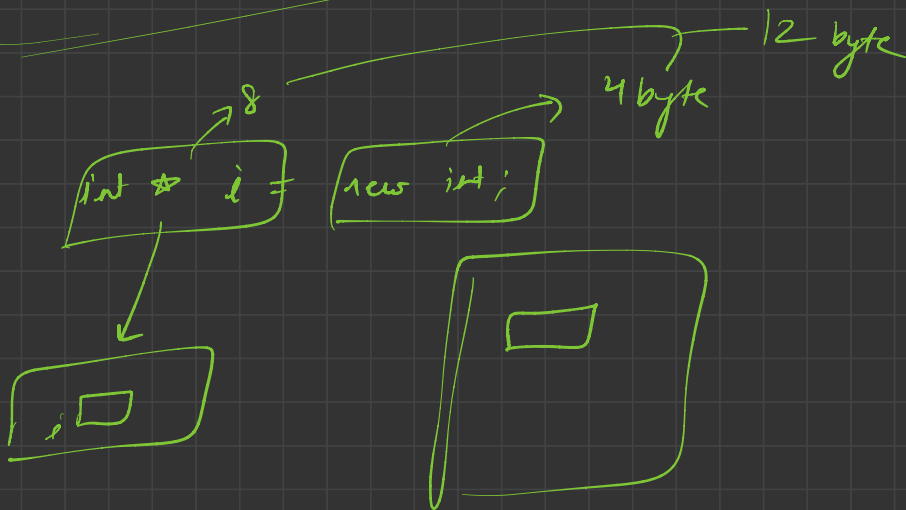
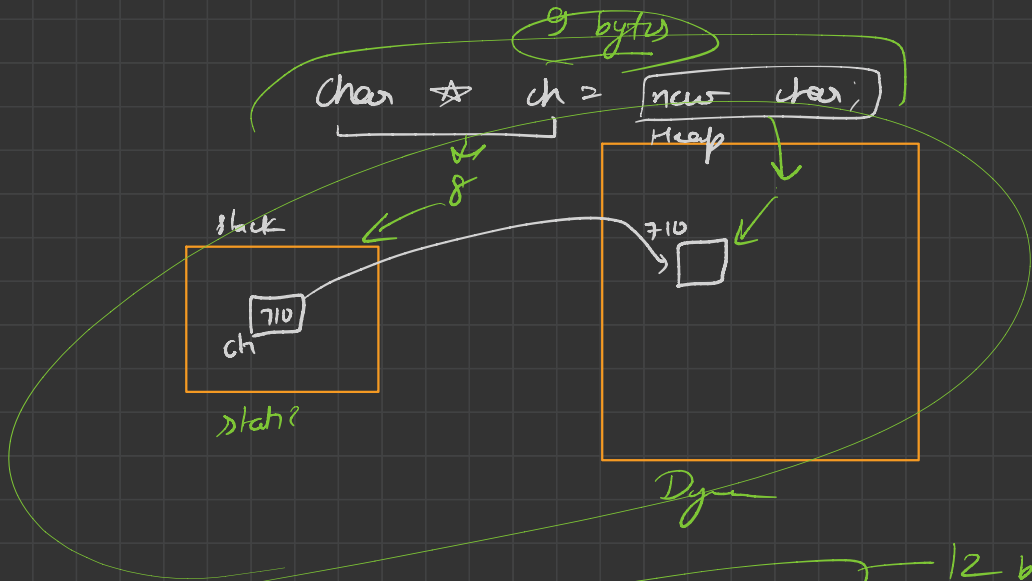


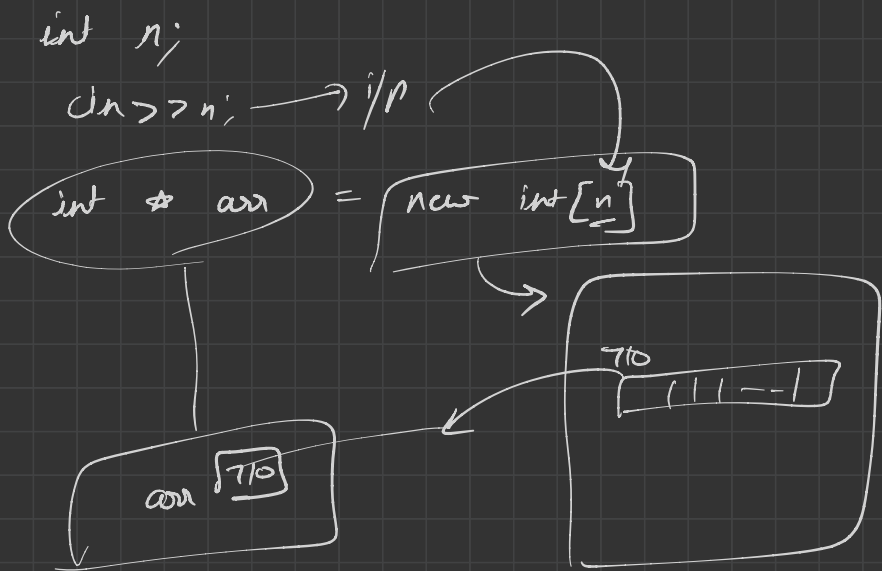
Heap memory

new keyword

char

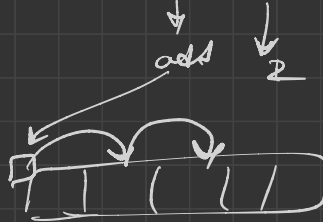






~~★~~

$$arr[i] = * (arr + i)$$



Diff b/w :-

Static

vs

Dynamic

①

int arr[50]

↓
 $50 \times 4 = 200 \text{ byte}$
↓
Stack

int *arr = new int[50]
↓ ↓
8 byte $50 \times 4 \rightarrow 200$ (heap)
↓
Stack ↘
 208

Case -> 1

while (true)
{
 int i = 5; → 1 int memory
 4 bytes
}

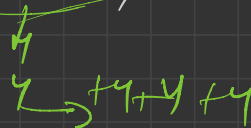
Car1-2

```
while (true)
```

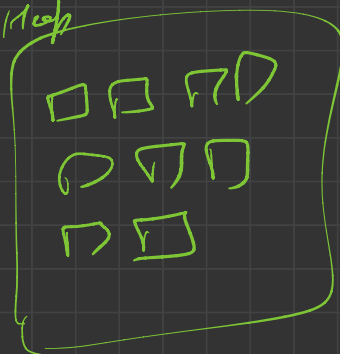
```
{
```

```
int * p = new int;
```

```
}
```



Heap



Program crash

static allocation

memory automatically
release

Dynamic allocation

manually release

"delete"

keyword

int

int* i = new int

for single
element
deletion

delete i

int arr[n]

arr

int* arr = new int[50];

for array
deletion

delete [] arr;